# AI SalesGrid
# Your AI Superpower in Sales for Enterprise

## On AWS

**The Problem:** B2B sales take weeks of back-and-forth emails and multi asking questions to align on pricing, rules, and proposals.

In large companies:

- Sales promise features that don't exist
- Offers violate company pricing rules
- Proposals don't follow legal/financial constraints
- Managers don't have real-time visibility
- CEO gets incomplete reports
- Everything is manual, slow, and inconsistent

This causes:

- Revenue leakage
- Legal risk
- Pricing mistakes
- Lost deals
- Internal conflict

**The Solution:** A multi-agent system where a client or sales rep talks to a **Amazon Nova and AWS Bedrock** that orchestrates 5+ background agents to validate, price, and report in seconds.

# Real-Time AI Multi-Agent Sales Orchestrator

## Your AI Superpower in Sales – Real-Time Enterprise Deal Engine

A voice-enabled, multi-agent system where:

- Sales talks naturally (can interrupt)
- AI validates offers instantly
- AI builds proposals
- AI checks rules automatically
- AI generates executive reports
- Everything is real-time

Powered by:

- Amazon Nova (voice + interruption)
- AWS Bedrock multi-agent orchestration
- AWS Bedrock Flow
- AWS hosting
- Knowledge Base (Bedrock Knowledge base/ OpenSearch)
- AWS API Gateway
- AWS Lambda
- AWS ECS

# Multi-Agent Architecture

## 0 Head Agent: Account Manager AI (Controller)

**Role:** Central brain

- Talks to user (voice)
- Delegates to other agents
- Controls workflow
- Handles interruptions
- Maintains deal context

Built with:

- AWS Bedrock (real-time conversation)
- Agents in Bedrock for orchestration

---

## 1 Solution Architect Agent (Question Analyzer)

"Agent for questioner to provide solution from what we have and send to supervisor"

**Role:**

- Understand client requirements
- Match with internal services
- Check feasibility
- Create technical solution summary
- Send to Supervisor Agent

**Uses:**

- Company services knowledge base
- Previous case database

---

## 2 Pricing & Offer Validation Agent

"Agent to make offers and validate with company rules"

**Role:**

- Generate pricing model
- Apply:
    - Discount limits
    - Margin thresholds
    - Country pricing policies
    - Compliance rules
- Reject or approve offer

**Example Rules:**

- Discount > 20% → needs approval
- Margin < 35% → reject
- Special pricing requires manager

Stored in:

- Dynamodb rule engine
- Or Redshift rule tables

---

# 3 Proposal Generator Agent

"Agent to make proposal for client with company rules"

**Role:**

- Generate:
  - Scope of work
  - Architecture summary
  - Timeline
  - Pricing table
  - Legal clauses
- Ensure:
  - Terms match company policy
  - No unsupported promises
  - Correct SLA

Outputs:

- PDF (Lambda + Python)
- Google Docs auto-generated

---

# 4 Executive Reporting Agent

"Agent for report for CEO and Heads"

**Role:**

- Generate:
  - Deal size
  - Risk score
  - Margin analysis
  - Win probability
  - Resource impact
- Summarize in:
  - Executive format
  - Dashboard format

Push to:

- AWS Redshift

- AWS RDS

---

## 5 Summarizer Agent

"Agent for Summarizes"

**Role:**

- Summarize:
  - Conversation
  - Proposal
  - Risk
  - Technical scope
- Create:
  - 1-minute executive summary
  - Email summary
  - CRM update text

---

## 6 Risk Assessment Agent

- Detect legal risk
- Detect overcommitment
- Flag unsupported features
- Score risk 1–10

---

## 7 Compliance Agent

- Ensure:
  - GDPR
  - Data residency
  - Security controls
- Especially important for big companies

---

## 8 Resource Capacity Agent

- Checks:
  - Are engineers available?
  - Is timeline realistic?
- Connects to HR data or mock DB

---

# AI Multi-Agent Account Manager

This solution introduces an **AI-powered Multi-Agent Account Manager** built on **AWS** that assists sales teams by automatically analyzing deals, generating pricing, assessing risk, preparing proposals, and creating executive summaries in real time using voice interaction.

The system is designed to work across multiple industries such as **Real Estate, SaaS, Finance, Insurance, and Consulting, IT Services**, while the demonstration use case focuses on **Real Estate Deal Management**.

---

# 1. Business Problem

In industries like **Real Estate**, sales teams spend significant time:

• Understanding client requirements
• Evaluating property deals
• Preparing pricing structures
• Assessing risk factors
• Creating proposals
• Writing executive reports

These activities are often **manual, slow, and fragmented across tools**.

This project introduces an **AI Account Manager Agent** capable of orchestrating specialized agents to automate the entire process.

---

# 2. Solution Overview

The solution uses **AWS AI services** to build a **voice-enabled AI account manager** capable of orchestrating multiple specialized agents.

The system allows a **sales executive to talk to the AI assistant**, which then coordinates several AI agents to analyze the deal and generate deliverables.

### Example conversation

User says:

"Client wants a 3-building residential complex investment in Dubai with a budget of $12M. Prepare a proposal."

The AI will automatically:

1□ Analyze client requirements
2□ Generate deal pricing
3□ Evaluate risks
4□ Create a formal proposal
5□ Generate an executive report
6□ Save the deal to CRM

---

# 3. High Level Architecture

The system consists of the following main layers:

### 1 User Interface Layer

A **Web Application** where users interact with the AI using voice.

**Technology**

• React / Next.js
• WebSocket audio streaming
• Real-time UI updates

The frontend streams audio to the AI system using **Streamlit or Amazon NOVA Act**.

---

### 2 Real-Time AI Interaction Layer

The **AWS Bedrock with Amazon Nova with AWS API Gateway** processes real-time voice interaction.

Capabilities include:

• Speech recognition
• Natural language understanding
• Interruptible responses
• Streaming responses

This enables **real-time conversation between the user and the AI agent**.

---

### 3 AI Orchestration Layer

The core of the solution is the **Account Manager Agent**, built using:

**AWS Bedrock (Agent + Flow)**

This component acts as the **Head Agent** responsible for coordinating specialized agents.

Responsibilities include:

• Understanding the user request
• Selecting the correct agents
• Orchestrating agent workflows
• Aggregating responses

---

# 4. Multi-Agent System

The system consists of **six specialized agents**.

---

# 1□ Solution Architect Agent

Role:
Analyzes the client requirements and defines the best solution approach.

Responsibilities:

• Understand customer needs
• Recommend property investment strategies
• Define deal structure
• Identify required services

Example output:

"Client requires a mixed-use residential investment project with high ROI potential."

---

# 2□ Pricing Agent

Role:
Generates financial estimates for the deal.

Responsibilities:

• Calculate property value
• Estimate ROI
• Generate pricing breakdown
• Compare market benchmarks

Data sources:

• Market price datasets
• CRM deal history
• Real estate price indexes

---

# 3□ Risk Agent

Role:
Analyzes financial and operational risks.

Responsibilities:

• Market volatility analysis
• Investment risk scoring
• Compliance verification
• Deal viability assessment

Example risk analysis:

• Market demand risk
• Regulatory risk
• Financial exposure

---

## 4□ Proposal Agent

Role:
Generates a **formal proposal document**.

Outputs:

• Investment proposal
• Property portfolio overview
• Financial projections
• Terms and conditions

Generated formats:

• PDF proposal
• Email-ready document

---

## 5□ Executive Report Agent

Role:
Generates **high-level reports for decision makers**.

Outputs include:

• Executive summary
• Key insights
• Investment opportunity overview

This is typically sent to **company leadership**.

---

## 6□ Summarizer Agent

Role:
Consolidates the outputs of all agents.

Responsibilities:

• Generate final summary
• Create action items
• Suggest next steps
• Prepare CRM updates

---

# 5. Knowledge Base & Grounding

The system integrates **AWS Bedrock Knowledge Base** to ground responses in enterprise data.

Data sources include:

• Company documents
• Real estate market reports

• Pricing templates
• Policies
• Proposal templates

This prevents hallucinations and ensures **accurate AI responses**.

---

# 6. Data Layer

The architecture uses AWS data services.

### RDS – OpenSearch – S3

Used for:

• Active deal storage
• Conversation history
• Agent interaction logs

---

### Redshift

Used for:

• Historical deal analysis
• Sales performance analytics
• Market intelligence

---

# 7. CRM Integration

The system connects with CRM platforms such as:

• Salesforce
• HubSpot
• Zoho CRM

Functions include:

• Loading active deals
• Updating deal progress
• Storing proposals
• Logging AI recommendations

This allows the AI to operate as a **virtual sales assistant**.

# 8. Deployment Architecture

The entire solution is hosted on **AWS**.

Core services include:

| Service | Purpose |
|---|---|
| AWS Lambda and ECS | Deploy AI agent backend |
| AWS Bedrock | AI models and agents |
| AWS Nova | Real-time voice interaction |
| AWS Bedrock KB | Knowledge base grounding |
| Dynamodb | Transactional data |
| Redshift | Analytics |
| AWS S3 | Document storage |

This architecture provides:

- High scalability
- Managed infrastructure
- Low operational overhead

---

# 9. End-to-End Workflow

The system operates in the following sequence:

### Step 1

User speaks through the web application.

### Step 2

Audio is streamed to **AWS API Gateway Connect with AWS Bedrock**.

### Step 3

Gemini processes speech and forwards the request.

### Step 4

The request reaches the **Account Manager Agent**.

### Step 5

The Head Agent orchestrates the sub agents:

- Solution Architect Agent
- Pricing Agent

• Risk Agent
• Proposal Agent
• Executive Report Agent

**Step 6**

Agents retrieve relevant knowledge using:

AWS Bedrock.

**Step 7**

Deal data is retrieved from:

Dynamodb / RDS / CRM.

**Step 8**

Results are consolidated by the **Summarizer Agent**.

**Step 9**

Outputs are generated:

• Proposal document
• Executive report
• CRM updates
• Real-time response to the user.

---

# 10. Security Considerations

Security measures include:

• IAM role-based access control
• Data encryption at rest and in transit
• Secure API authentication
• Audit logging via Cloud Trail

---

# 11. Scalability

The architecture is designed for high scalability.

Benefits include:

• Serverless deployment via Lambda and ECS
• Automatic scaling based on traffic
• Multi-agent parallel processing
• Real-time streaming responses

# 12. Industry Adaptability

Although the demonstration focuses on **Real Estate**, the platform is designed to support multiple industries.

Examples:

| Industry | Use Case |
|---|---|
| Real Estate | Property deal analysis |
| Finance | Investment evaluation |
| Insurance | Policy risk analysis |
| SaaS | Enterprise sales assistant |
| Consulting | Proposal automation |
| IT Services | Proposal and provide services |

# 13. Expected Business Impact

This system can deliver significant improvements:

- 60–80% faster proposal generation
- Automated deal analysis
- Reduced manual workload
- Improved sales decision making
- Better CRM data quality

# Multi-Agent Flow

User (voice)
↓
Account Manager Agent
↓
Solution Architect Agent
↓
Pricing Agent
↓
Risk Agent
↓
Proposal Agent
↓
Executive Report Agent
↓
Summarizer Agent

# System Architecture (AWS)

### Brain Layer

- AWS API Gateway
- AWS Bedrock for multi-agent routing with Amazon Nova

### Knowledge Base

- S3 (rules)
- RDS (pricing models)
- S3 (documents)

### Backend

- Lambda (microservices per agent)
- AWS API Gateway (agent communication)
- Lambda (trigger workflows)

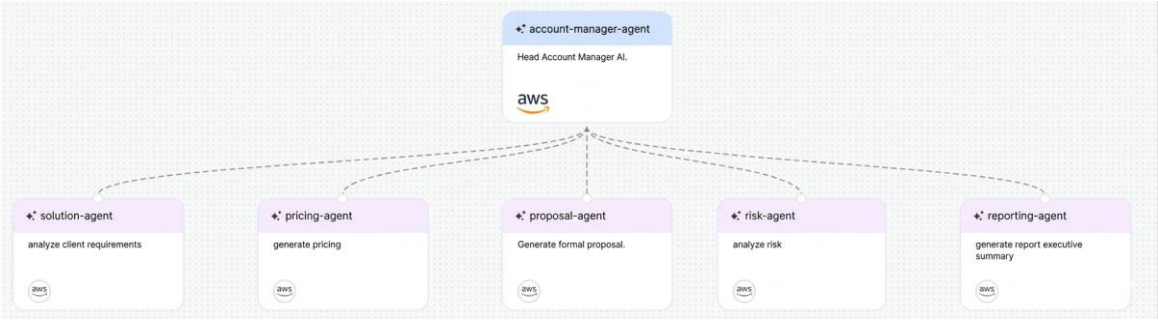
### Reporting

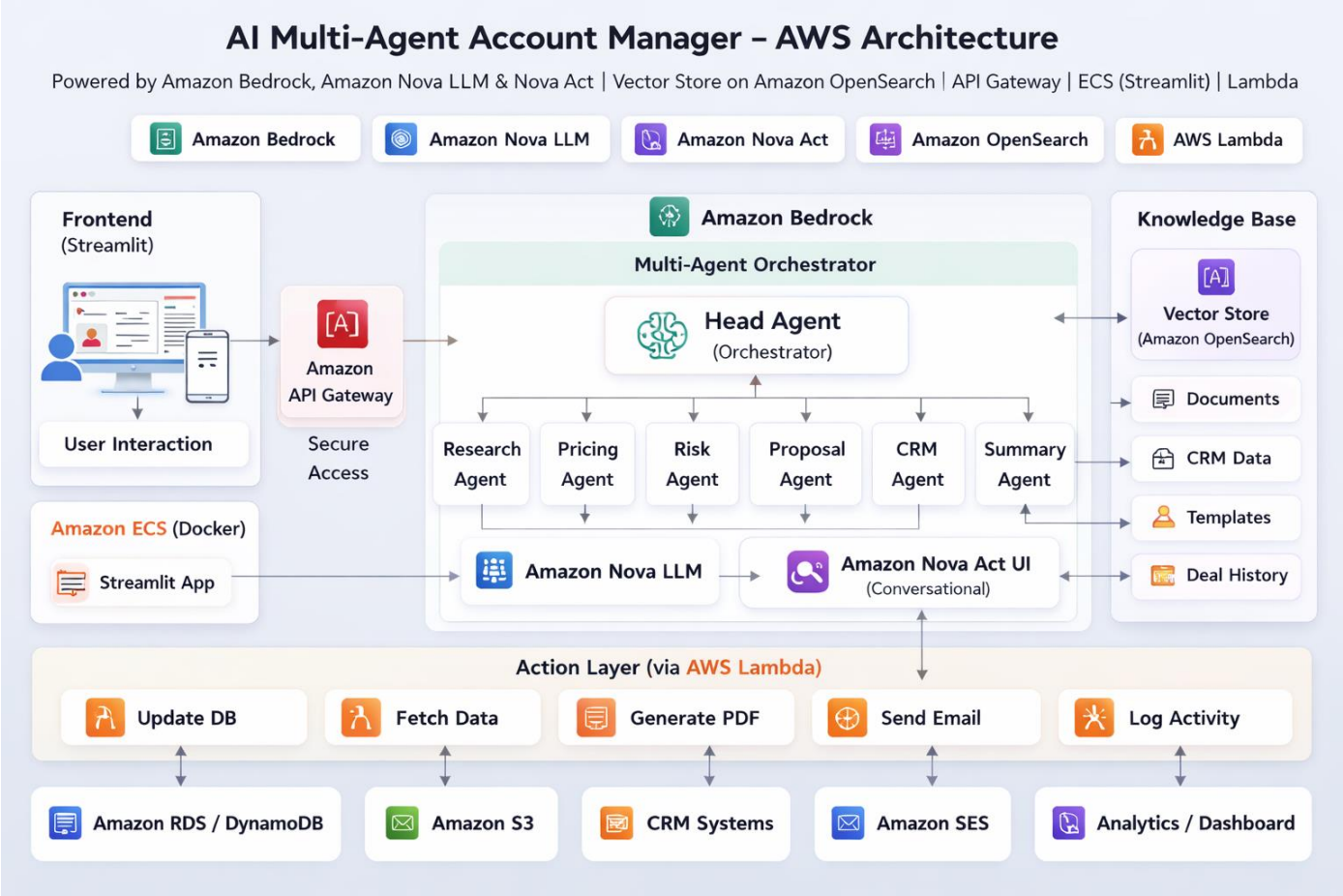
- Redshift
- Dynamodb

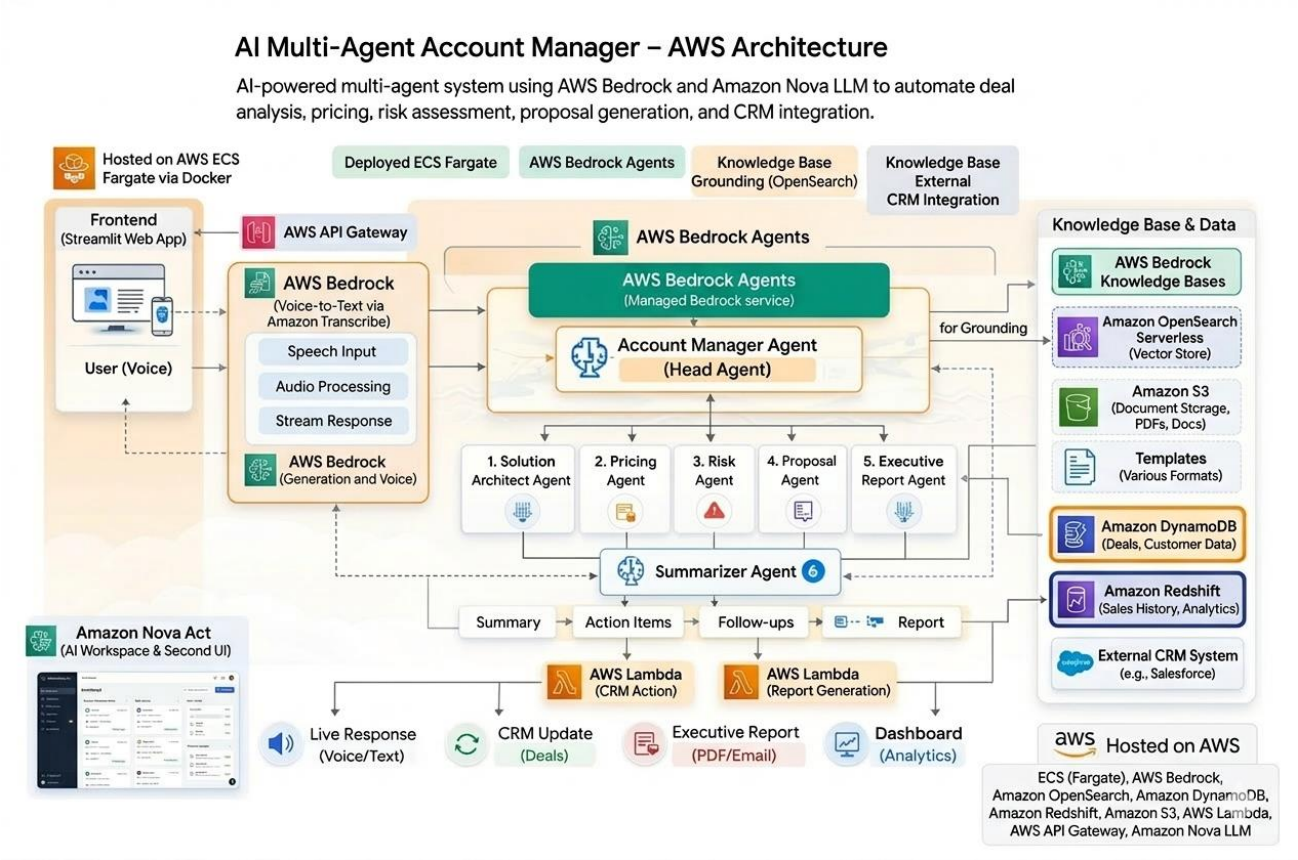
# FINAL ARCHITECTURE OVERVIEW

Frontend (Web App)
↓ (WebSocket Audio Stream)
Streamlit or Amazon Nova Act
↓
Account Manager Agent (AWS API Gateway + AWS Bedrock + Amazon Nova)
↓
Sub Agents (Pricing / Proposal / Risk / Reporting)
↓
AWS Bedrock (Knowledge Base grounding)
↓
RDS / Dynamodb / Redshift /  OpenSearch (Deal storage)

# ARCHITECTURE






This architecture component diagram details a fully managed solution on AWS

**AWS Multi-Agent AI Architecture**

This architecture component diagram illustrates a fully managed AI solution on **AWS** that integrates a **multi-agent orchestration framework powered by foundation models** to automate account management tasks such as deal analysis, pricing evaluation, proposal generation, and CRM integration.

Key features of this system include:

**• AI-Driven Interaction Layer**

The system enables conversational and text-based interaction through a web interface built with Streamlit and deployed in containers on **Amazon ECS**.
User requests are securely routed through **Amazon API Gateway**, which exposes REST or WebSocket endpoints that connect the frontend with backend AI agents.

**• Multi-Agent Orchestration**

A **Head Agent** orchestrates multiple specialized AI agents using **Amazon Bedrock** with **Amazon Nova LLM models**.
The head agent coordinates tasks and delegates them to domain-specific agents including:

- Solution Architect Agent
- Pricing Agent
- Risk Assessment Agent
- Proposal Generation Agent
- CRM Agent
- Executive Report Agent

Each agent independently processes its task while contributing results to the overall workflow.

**• Knowledge Base Grounding (RAG)**

Agents retrieve contextual enterprise knowledge using **retrieval-augmented generation (RAG)**.
Documents such as company policies, pricing rules, templates, and CRM data are embedded and stored in a **vector database on Amazon OpenSearch Service**.

This enables agents to query relevant information dynamically and generate accurate responses grounded in company knowledge.

**• Action & System Integration Layer**

Agents trigger operational workflows through **AWS Lambda**, enabling the system to interact with backend services such as:

- Updating CRM or deal databases
- Fetching customer or sales history
- Generating reports or proposals
- Sending notifications or emails
- Logging activity and analytics

Lambda functions provide a scalable serverless execution layer that connects AI reasoning with real business actions.

**• Multi-Channel AI Interface**

In addition to the Streamlit frontend, the architecture integrates **Amazon Nova Act** as a conversational interface for advanced AI interactions and automation workflows.

**• Post-Processing & Reporting**

A **Summarizer Agent** consolidates the outputs from all specialized agents into structured outputs including:

- Deal summaries
- Recommended actions
- Follow-up tasks
- Executive reports

These outputs can be delivered through dashboards, exported reports, or CRM updates.

---

**Technical Implementation on AWS**

**• Orchestration**

Use **multi-agent orchestration logic built on Amazon Bedrock** with **Amazon Nova foundation models** to coordinate agent collaboration and reasoning workflows.

**• Knowledge Base**

Store enterprise documents in **Amazon S3**, generate embeddings, and index them in **Amazon OpenSearch Service** for vector search and RAG retrieval.

**• API Layer**

Expose backend AI capabilities through **Amazon API Gateway**, enabling secure communication between the frontend application and the AI agents.

**• Actions & Automation**

Use **AWS Lambda** to execute operational tasks triggered by agents, including database updates, integrations with CRM systems, and report generation.

**• Hosting**

Deploy the frontend **Streamlit application** in Docker containers on **Amazon ECS** for scalable containerized hosting.

**Step-by-Step Execution Summary**

**1. Data Preparation**

- Upload company knowledge documents (PDFs, policies, pricing rules) to **Amazon S3**.

**2. Knowledge Base Setup**

- Generate embeddings and index documents into **Amazon OpenSearch Service vector store**.

**3. Agent Framework**

- Build the **multi-agent orchestration system using Amazon Bedrock and Amazon Nova models**.
- Implement the Head Agent and specialized task agents.

**4. Backend API**

- Create API endpoints using **Amazon API Gateway** to allow the frontend to communicate with AI agents.

**5. Action Functions**

- Implement business logic using **AWS Lambda functions** to perform system actions (DB updates, CRM integration, report generation).

**6. Frontend Deployment**

- Containerize the Streamlit UI and deploy it on **Amazon ECS**.

**7. AI Interaction**

- Integrate **Amazon Nova Act** as a conversational interface for advanced AI interactions and automation workflows.

# AI SalesGrid
# Your AI Superpower in Sales for Enterprise

**On AWS**

Developed by: Fady Nabil
https://github.com/fadynabil10